



Robert Hendrich, Lukas Pilz und Leon Frey

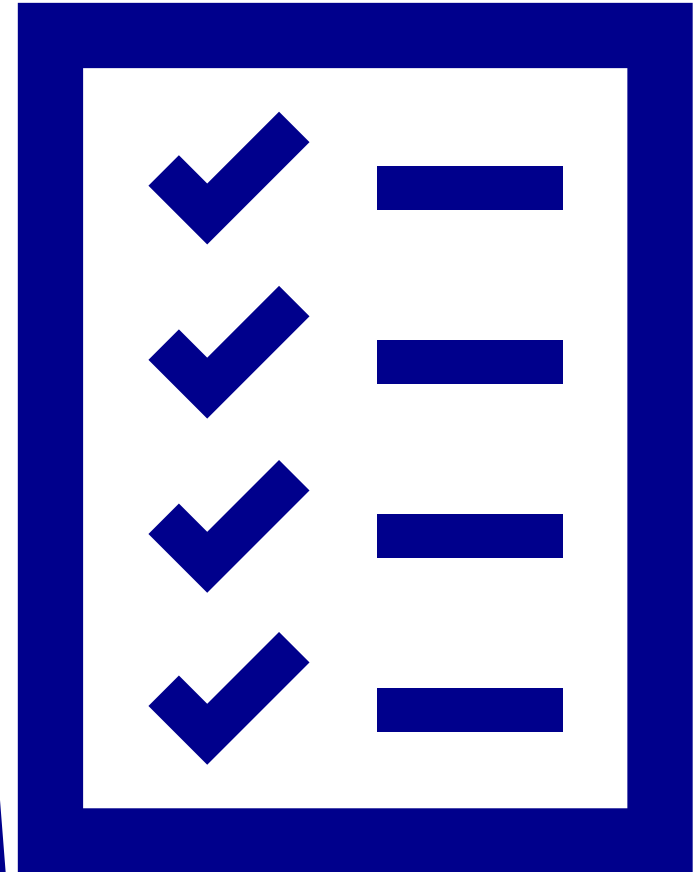
Dresden, 23. Juni 2026

Gestaltungsansätze der Advanced Data Analytics

Group 3: Cycling Stage Prediction

Gliederung

1. Aufgabenstellung
2. Data Engineering und Feature Selection
3. Univariate Analyse
4. Bivariate Analyse
5. Data Splitting
6. Implementierung der Machine Learning Modelle
 - 6.1 EBM
 - 6.2 XGBoost
 - 6.3 TabPFN
7. Ausblick



1. Aufgabenstellung

Topic 1: Cycling stage prediction

Ziel: Vorhersage der Top-5/Top-10/Top-20-Platzierten je Etappe (Fahrerranking nach Abschlussplatzierung)

- 1) **Data collection, integration, and cleaning** (e.g., filtering, harmonization, noise removal, removal of duplicates, missing values, feature scaling, etc.)
- 2) **Descriptive data analysis for data exploration & data understanding** (e.g., dataset properties, distribution, univariate, bivariate, multi-variate, correlations, imbalance, etc.)
- 3) **Development of an automated machine learning pipeline** (feature preparation, sampling/data splitting, model training, evaluation strategy, etc.)
- 4) **Implementation of different machine learning models**
 - **Simple models** (e.g., *linear models, decision tree*)
 - **Advanced flexible models** (e.g., *tree-based ensembles, deep neural networks, tabular foundation models*)
 - **Advanced interpretable models** (e.g., *generalized additive models*)
- 5) **Model evaluation, diagnosis, interpretation** (post-hoc explanations, interpretable model structure)





2. Data Engineering und Feature Selection

2. Data Engineering und Feature Selection



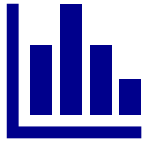
- **NA handling + Data cleaning**
- **Feature Engineering**
 - Alter am Renntag
 - BMI (Größe und Gewicht)
 - Wetterdaten transformiert
 - Mittelwerte (Start/Ziel)
 - Windindex (Windgeschwindigkeitsänderung + Windrichtungsänderung $[0,1]$)
- **Neues Feature**
 - Steigung auf dem letzten km -> PCS Website
- **Eliminierung Data Leakage**
 - Feature Won-How
 - Transformation der Ranglisten- und Teamstärkefeatures



3. Univariate Analyse

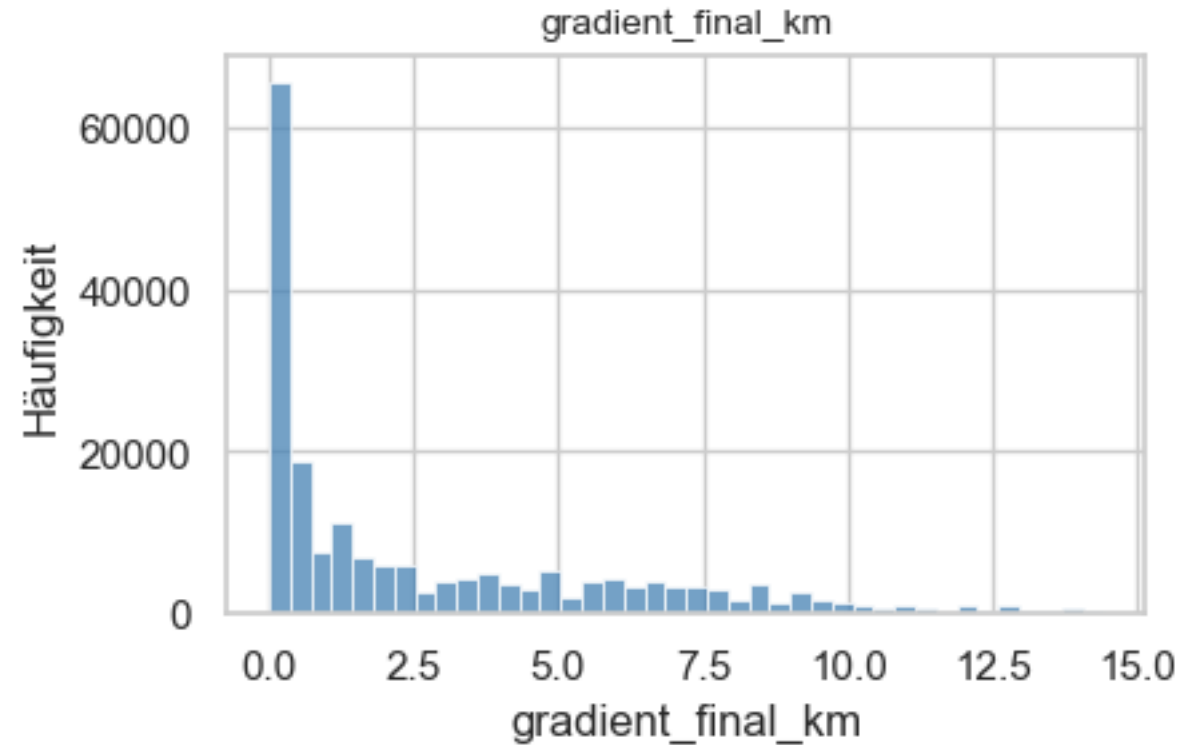
Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

Neue Features

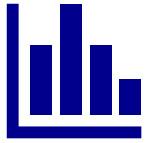


gradient_final_km

- Median: nur 1,2
- Mittelwert: 2,8
- Max: 14,4
- Skew: 1,224

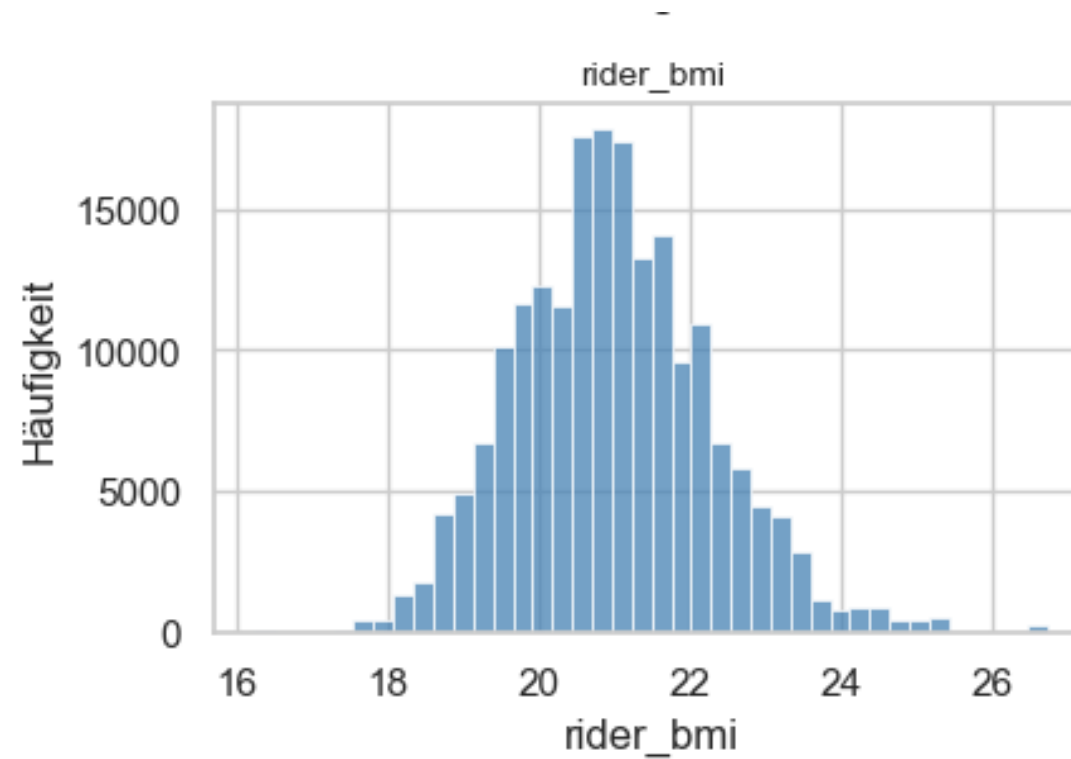


Neue Features

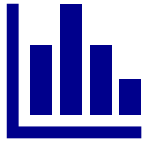


rider_bmi

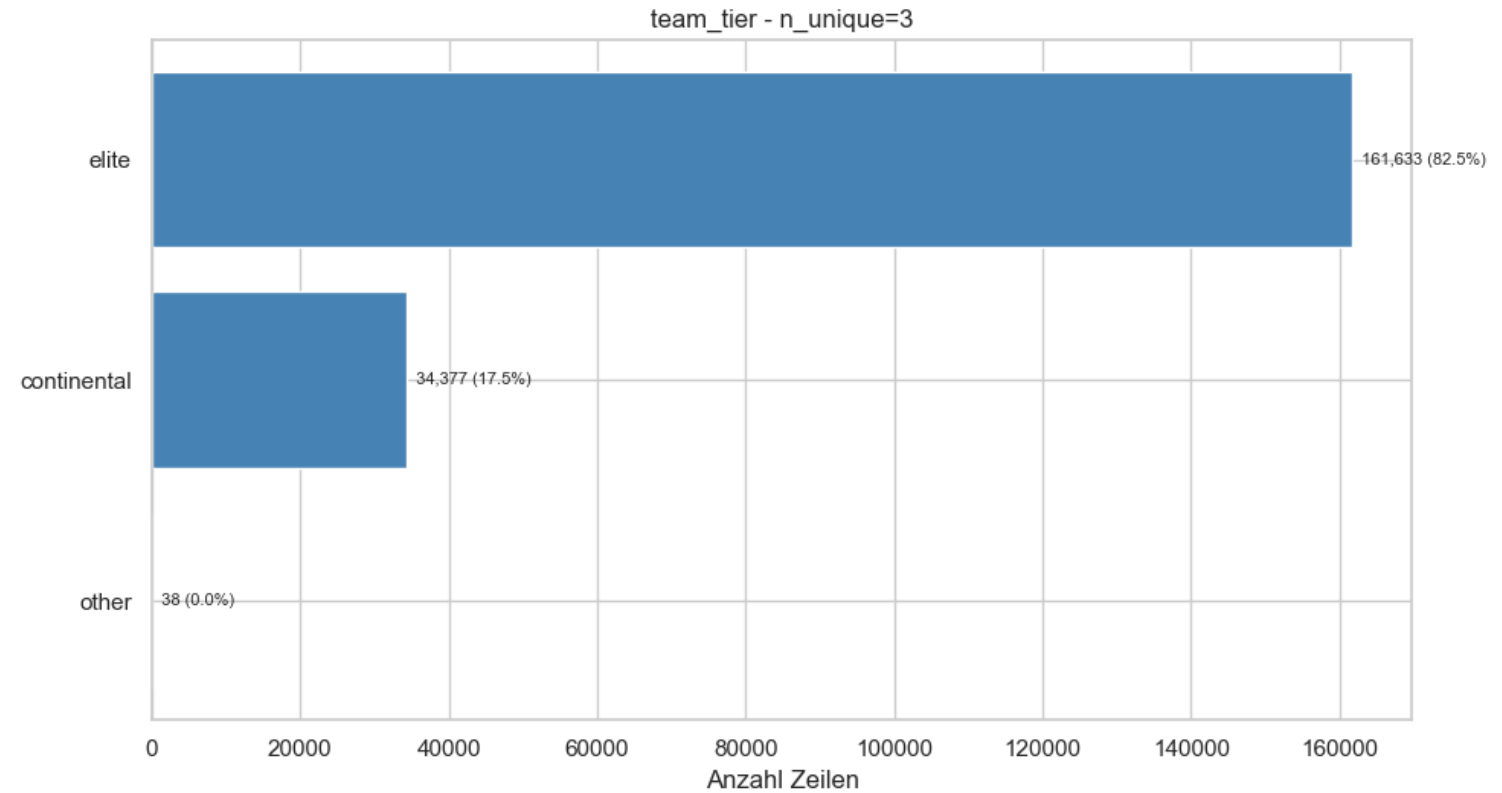
- Median: 20,91
- Mean: 20,99



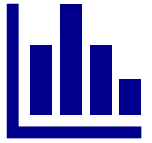
Neue Features



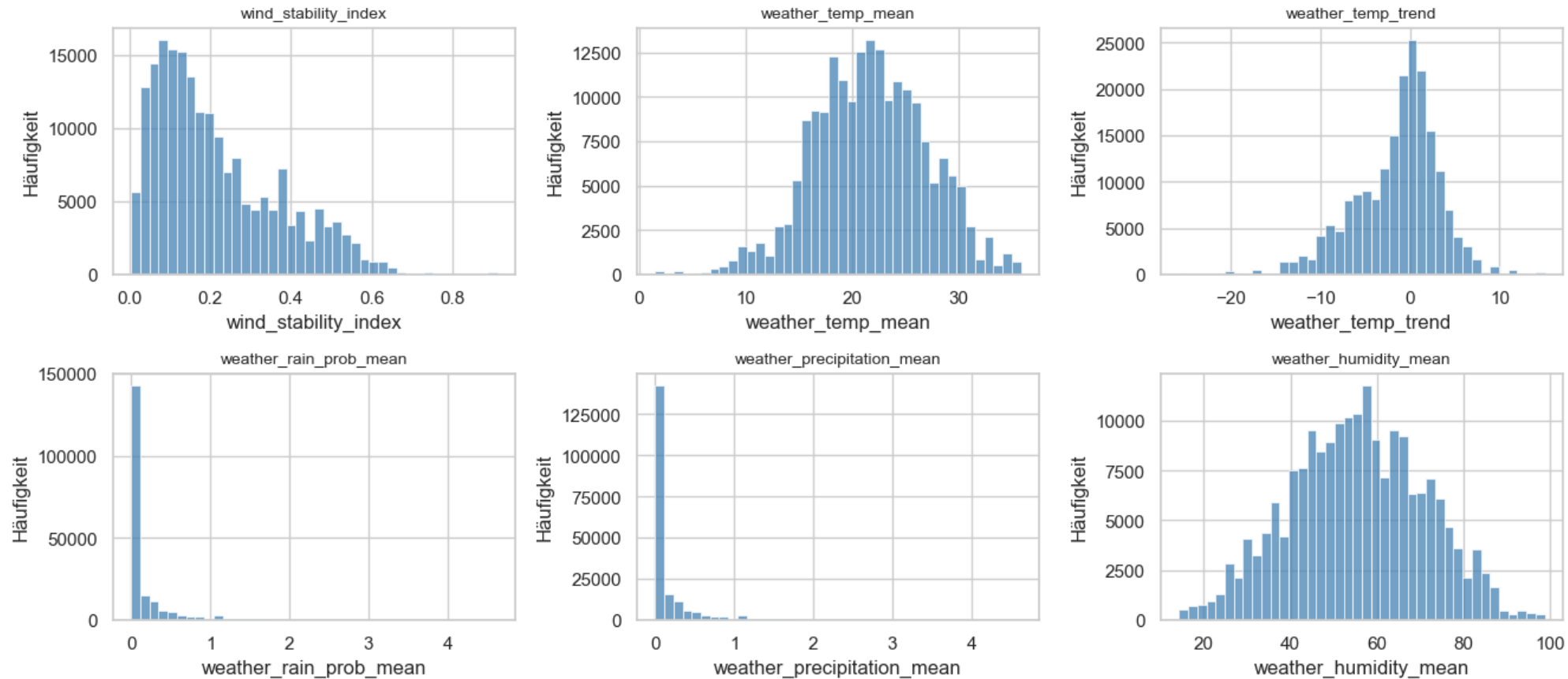
team_tier aus team_class



Neue Features



Wetterfeatures - Histogramme

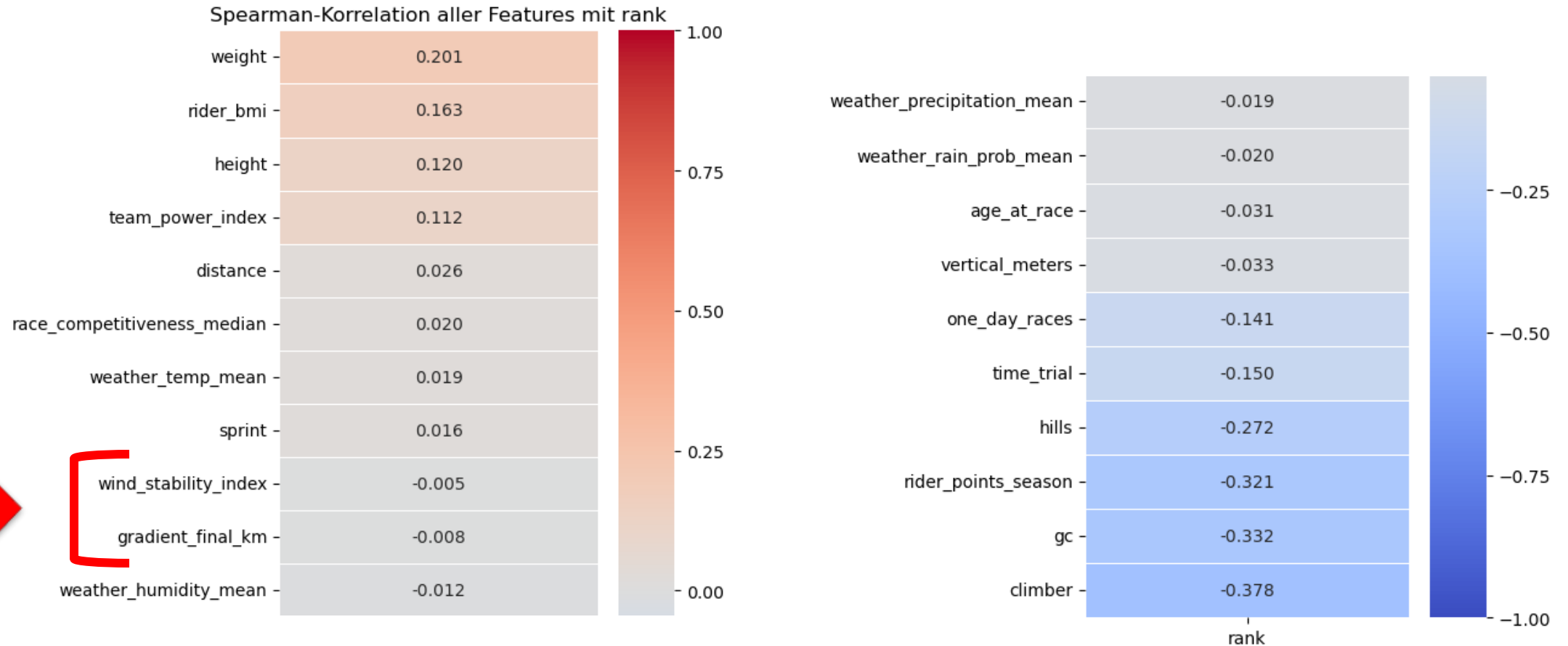




4. Bivariate Analyse

Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

Korrelation der metrischen Variablen mit der Zielvariable „rank“





5. Data Splitting

Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

5. Data Splitting



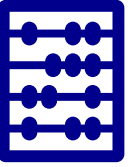
- Eindeutige Stage ID erzeugt (Rennen, Jahr und Etappe)
- Ordinale Zielvariable erzeugt
 - 3 = Top 5, 2 = Top 10, 1 = Top 20, 0 = Rest
 - Grund: Unterschied zwischen Platz 1 und 2 vs. Platz 80 und 81
 - Trainingsdaten ≤ 2022
 - Valdierungsdaten = 2023
 - Testdaten ≥ 2023



6.1 EBM Modell (Explainable Boosting Machine)

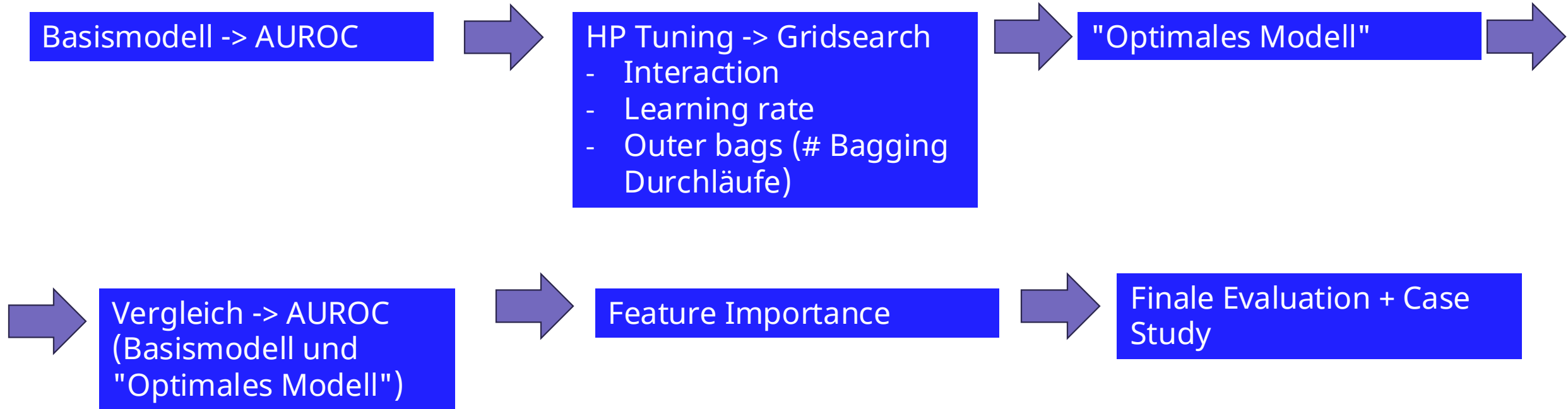
Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

6.1 EBM Modell

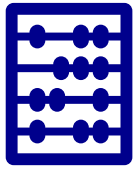


- Transparentes Startmodell -> Glassboxmodell
- Hier als Zielvariable ordinal Codierten Rankingklassen (Top 5 (3), 10 (2), 20 (1), Rest (0))
 - Abstand Ränge nicht immer gleich "wichtig/ sinnvoll" (Platz 1/2 vs. Platz 80/81)
- Ziel:
 - Featureeffekte verstehen
 - Ranking aus den Wahrscheinlichkeiten für die Rankingklassen erstellen
 - McRank Verfahren (Liu 2009)
 - Schätzung der Klasse inkl. Ranking über den Expected Relevance Score
 - $\text{Score} = P(k=3) \times 3 + P(k=2) \times 2 + P(k=1) \times 1 + P(k=0) \times 0$

6.1 EBM Modell



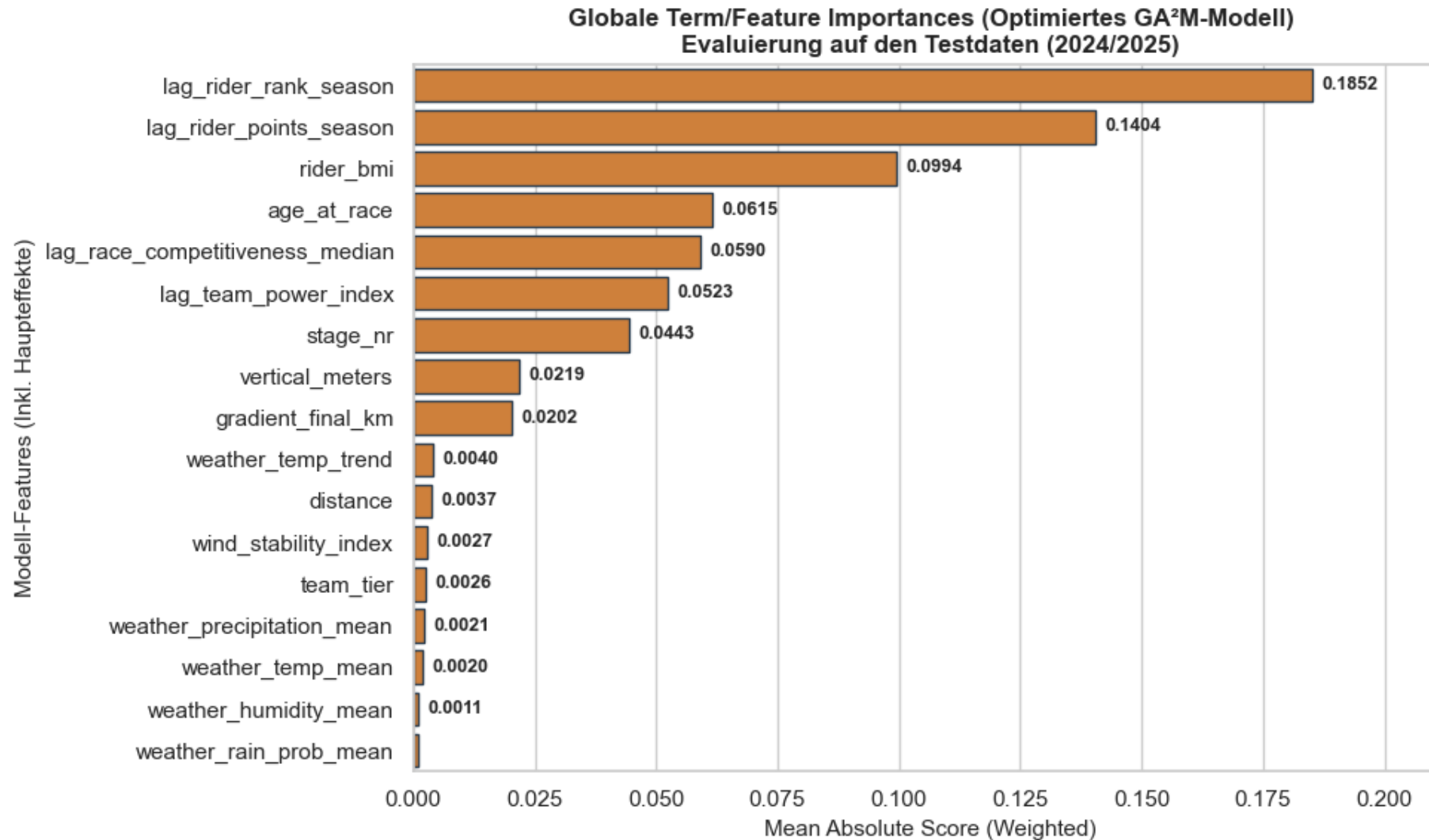
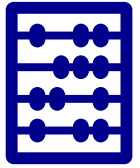
6.1 EBM Modell – Evaluation Tuning Modell



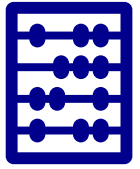
Metrik	EBM Baseline (1D)	EBM Tuned (GA²M)
ROC-AUC (Top10)	0.7281	0.7698
NDCG@5	29.81%	38.84%
NDCG@10	29.54%	37.40%
NDCG@20	32.11%	39.49%
Winner Hit@1	15.32%	18.92%
Winner Hit@5	41.44%	43.24%
Winner Hit@10	53.15%	56.76%
Winner Hit@20	61.26%	63.96%
Spearman Rho	0.2108	0.2858

Score = $P(k=3) \times 3 + P(k=2) \times 2 + P(k=1) \times 1 + P(k=0) \times 0$
McRank Verfahren (Liu 2009)

6.1 EBM Modell – Feature Importance



6.1 EBM Modell – Case Study (TdF 2025 E. 12)



Top 10 Fahrer sortiert nach EBM-Erwartungswert

Prognose-Rang	Fahrer	EBM-Score	Realer Rang
1	Tadej Pogačar	2.3357	1
2	Marc Hirschi	2.3324	57
3	Remco Evenepoel	2.3200	7
4	Ben O'Connor	2.1840	16
5	Biniam Girmay	2.1757	117
6	Jonas Vingegaard	2.1651	2
7	Carlos Rodríguez	2.1541	22
8	Lennert Van Eetvelt	2.1507	108
9	Matteo Jorgenson	2.1330	15
10	Mattias Skjelmose	2.1001	29

Realer Rang	Fahrer	EBM-Score	Prognose-Rang
1	Tadej Pogačar	2.3357	1
2	Jonas Vingegaard	2.1651	6
3	Florian Lipowitz	1.7067	35
4	Tobias Halland Johannessen	1.6821	38
5	Oscar Onley	2.0292	13
6	Kévin Vauquelin	1.5261	53
7	Remco Evenepoel	2.3200	3
8	Felix Gall	1.6291	43
9	Primož Roglič	1.8490	24
10	Cristián Rodríguez	1.5440	50



6.2 XGBoost

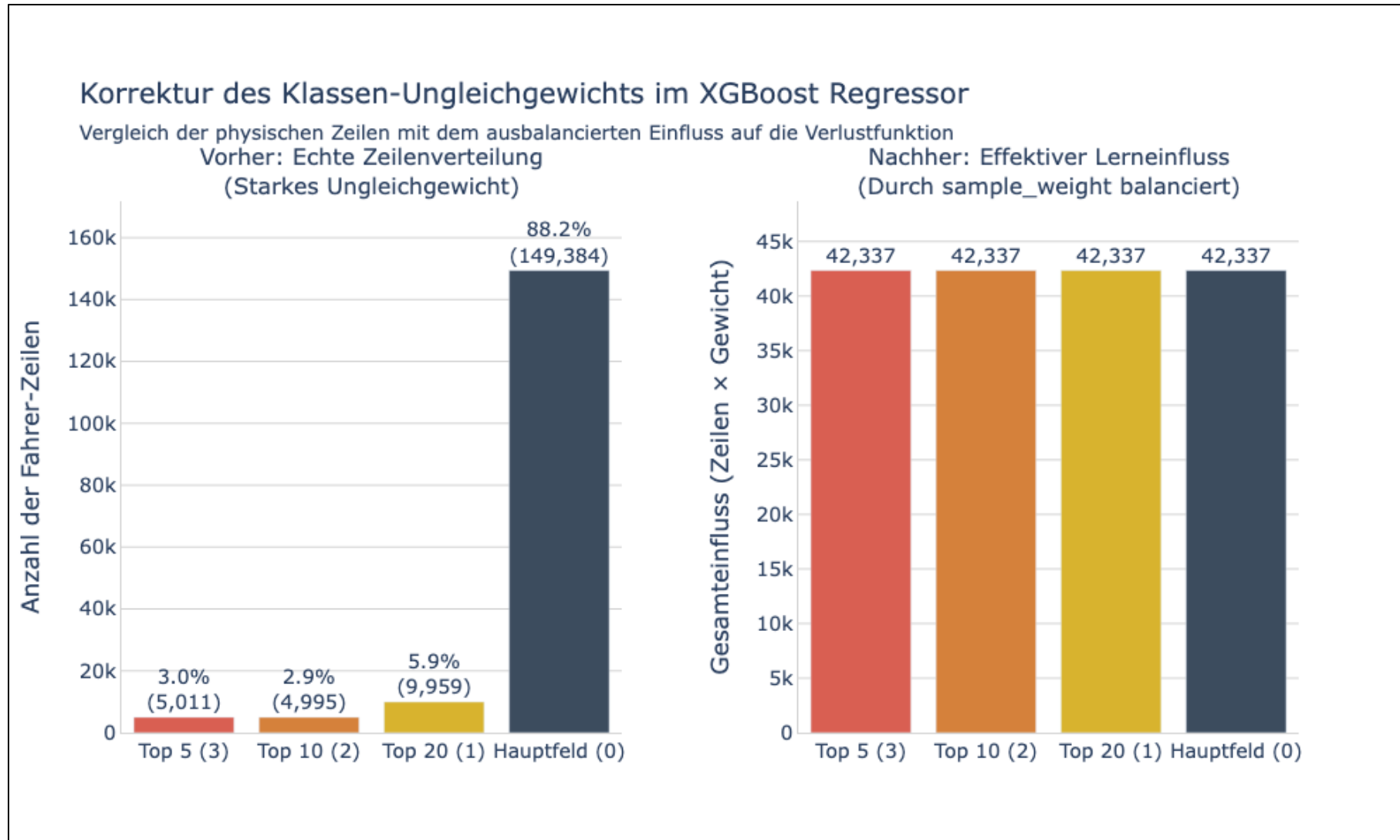
Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

6.2 XGBoost



- XGBoost als Verfahren des überwachten Lernens
 - Basis: Gradient Boosted Trees
- Zielvariable als ordinale Größe modelliert
 - Daher: Einsatz eines Regressors
 - Regressor bildet Rangordnung besser ab als Classifier
- Kontinuierliche Vorhersagen als Grundlage für das Ranking
- Anschließende Umwandlung in ein Ranking pro Stage

6.2 XGBoost



6.2 XGBoost



	meta_name	true_rank	predicted_rank	raw_prediction
4613	Michael Matthews	141.0	1.0	2.153720
4478	Mads Pedersen	4.0	2.0	2.131749
4612	Kaden Groves	140.0	3.0	1.993400
4495	Geraint Thomas	22.0	4.0	1.924386
4603	Alberto Dainese	131.0	5.0	1.916959
4501	Thymen Arensman	28.0	6.0	1.906357
4485	Lorenzo Rota	11.0	7.0	1.902703
4554	Vincenzo Albanese	81.0	8.0	1.901552
4497	João Almeida	24.0	9.0	1.866239
4518	Andreas Leknessund	45.0	10.0	1.842879
4523	Simone Consonni	50.0	11.0	1.838693
4492	Alberto Bettiol	19.0	12.0	1.836195
4512	Santiago Buitrago	39.0	13.0	1.829249
4558	Natnael Tesfatsion	85.0	14.0	1.821303
4511	Primož Roglič	38.0	15.0	1.819473
4615	Fernando Gaviria	143.0	16.0	1.795008
4488	Lennard Kämna	14.0	17.0	1.783890
4584	Jake Stewart	112.0	18.0	1.765831
4494	Damiano Caruso	21.0	19.0	1.761440
4506	Pavel Sivakov	33.0	20.0	1.757168

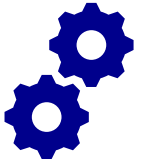
- $\emptyset$ NDCG@5: 0.4516
- $\emptyset$ NDCG@10: 0.4307
- $\emptyset$ NDCG@20: 0.4413



6.3 TabPFN

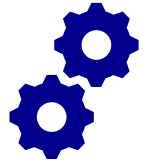
Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

6.3.1 Was ist TabPFN?



- (Tabular Prior-data Fitted Network)
- Auf Millionen von synthetischen Daten vortrainiert
 - Verständnis und Vorwissen zu Tabellendaten
- Transformer-Modell mit In-Context-Learning für Tabellen
 - neue Trainingsdaten werden als Kontext übergeben
- keine Hyperparameter-Anpassung notwendig, da bereits bei der Entwicklung trainiert
- sehr stark bei kleinen Datensätzen

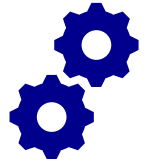
6.3.2 tabpfn (lokal) vs tabpfn-client (API)



- Problem, da In-Context-Learning sehr rechenaufwendig
- Lösung: API-Key für Prior Labs
 - Übernehmen GPU-Compute, Skalierung, Modellversionierung

Lokal vs API	Umfang Training (Kontext)	Umfang Test	GPU-Zeit
lokal	15.000 Zeilen	9.000 Zeilen	99 min
API	100.000 Zeilen	9.000 Zeilen	2 min

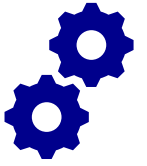
6.3.3 Nutzungslimits API



- Problem API Nutzungslimits
- Lösung: Herantasten mit `get_api_usage()`
- Vergleich Verbrauch Daily Limit (bei 9.000 Testzeilen)
 - $\text{tokens} = \max(5.000, (\text{train_rows} + \text{test_rows}) \times \text{columns} \times \text{n_estimators})$

Trainingskontext (Etappen)	TabPFN v2.x (Formel)	TabPFN 3.0 (sublinear)
25.093 (149)	4,62%	0,10%
50.007 (294)	8,01%	0,16%
75.030 (440)	11,41%	0,21%
100.139 (589)	14,83%	0,26%

6.3.4 Classifier vs Regressor

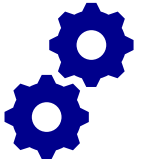


Setup:

- Trainings für Validation: meta_year $\leq$ 2022
- Validation: meta_year = 2023
- Finaler Test: meta_year $\geq$ 2024 (2023 im finalen Training)

- Imbalanced Data:
 - TabPFN Classifier: balance_probabilities = True
 - TabPFN Regressor: Interpretation als Scoring-Modell (sortieren innerhalb der Etappe)
--> keine echte Regression

6.3.4 Classifier vs Regressor

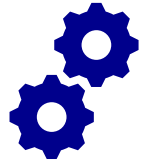


Classifier Rankings:

- Gesamtscore (Expected Relevance Score):
 - $P(Y=1) + 2 * P(Y=2) + 3 * P(Y=3)$
 - $P(Y=1) + 3 * P(Y=2) + 6 * P(Y=3)$
- "Wer kommt wahrscheinlich in die Top 5/10/20 ?"
 - Top 5: $P(Y=3)$
 - Top 10: $P(Y=3) + P(Y=2)$
 - Top 20: $P(Y=3) + P(Y=2) + P(Y=1)$

Regressor: Regressionsscore

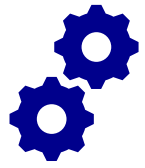
6.3.4 Classifier vs Regressor



Vergleich Classifier Scores vs. Regressionsscore

actual_train_rows	model_type	score_formula	ndcg_at_5	ndcg_at_10	ndcg_at_20	mean_stage_spearman
25093	Classifier	$P(1) + 2 \cdot P(2) + 3 \cdot P(3)$	0.391817	0.399167	0.431884	0.304621
25093	Classifier	$P(1) + 3 \cdot P(2) + 6 \cdot P(3)$	0.389949	0.393978	0.428552	0.302680
25093	Regressor	Regressionsscore	0.393191	0.412905	0.441389	0.309280
50007	Classifier	$P(1) + 2 \cdot P(2) + 3 \cdot P(3)$	0.382235	0.385198	0.420833	0.294577
50007	Classifier	$P(1) + 3 \cdot P(2) + 6 \cdot P(3)$	0.373420	0.378054	0.413487	0.291574
50007	Regressor	Regressionsscore	0.388207	0.402829	0.436574	0.309918
75030	Classifier	$P(1) + 2 \cdot P(2) + 3 \cdot P(3)$	0.384194	0.385371	0.429591	0.303959
75030	Classifier	$P(1) + 3 \cdot P(2) + 6 \cdot P(3)$	0.373758	0.380687	0.425530	0.301015
75030	Regressor	Regressionsscore	0.387995	0.411514	0.441099	<u>0.311970</u>
100139	Classifier	$P(1) + 2 \cdot P(2) + 3 \cdot P(3)$	0.382596	0.392759	0.436032	0.304674
100139	Classifier	$P(1) + 3 \cdot P(2) + 6 \cdot P(3)$	0.384850	0.389240	0.432998	0.302249
100139	Regressor	Regressionsscore	<u>0.400880</u>	<u>0.416142</u>	<u>0.444123</u>	0.310264

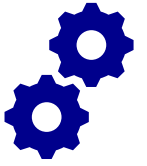
6.3.4 Classifier vs Regressor



Vergleich Top-n Wahrscheinlichkeiten mit Regressionsscore

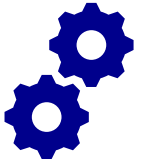
Metrik	Bestes Ergebnis Classifier	Bestes Ergebnis Regressor	Differenz
Precision@5	25,61% (25T)	25,61% (100T)	0,00%
Precision@10	32,81% (100T)	33,33% (100T)	-0,52%
Precision@20	40,35% (75T)	39,74% (100T)	0,61%
Recall@5	25,79% (100T)	25,79% (100T)	0,00%
Recall@10	32,85% (100T)	33,39% (100T)	-0,54%
Recall@20	40,39% (75T)	39,79% (100T)	0,60%
Top_5_Accuracy	38,60% (100T)	35,09% (25T)	3,51%
Top_10_Accuracy	52,63% (50T)	54,39% (100T)	-1,76%
Top_20_Accuracy	61,40% (25T)	61,40%	0,00%

6.3.5 Fazit TabPFN



- Mehr Trainingskontext verbessert Vorhersage
- Gesamtscores: Regressionsscore > Scores aus Classifier
- Top-n Zugehörigkeit: Classifier etwas besser

6.3.6 ToDo



- Einstellungen für TabPFNClassifier() und TabPFNRegressor()
 - n_estimators, softmax_temperatures, thinking_mode (API), thinking_metric (API)
- Festlegen Modell + Test für 2024 + 2025
- Interpretierbarkeit mit SHAP



7. Ausblick

Robert Hendrich, Lukas Pilz und Leon Frey
Dresden, 23. Juni 2026

7. Ausblick



- XGB Ranker implementieren
- Globaler Vergleich der Modelle (Evaluationsstrategie)